

HabloTruck Platform

Use Cases Guide

Subscription Journeys, Stripe and ManyChat

Documento operativo para QA, validacion integral y presentacion interna.

Cubre Stripe, Webhooks, Reducer, Access, ManyChat y Journeys de suscripcion.

Resumen visual

1. Stripe sigue siendo source of truth del lifecycle.
2. SubscriptionReducer sigue decidiendo transiciones reales.
3. ManyChat comunica estados; no decide acceso ni billing.

1. Alta Inicial de Suscripcion

Checkout inicial y activacion de acceso

Pasos del flujo

- ? El frontend o ManyChat solicita al backend Get Payment Link.
- ? El backend crea un Stripe Checkout Session de suscripcion con card y link.
- ? El usuario completa el pago en Stripe.
- ? Stripe envia checkout.session.completed al webhook.
- ? StripeSubscriptionHandler crea o actualiza al usuario y vincula StripeCustomerId y StripeSubscriptionId.
- ? AccessOrchestrator recomputa acceso y sincroniza ManyChat.
- ? Si todo es correcto, el usuario queda con acceso FULL.

Que validar

- Usuario creado o actualizado con StripeCustomerId y StripeSubscriptionId persistidos.
- Acceso efectivo Full.
- ManyChat sincronizado sin activar payment recovery ni journeys contradictorios.

2. Renovacion Automatica Normal

Ciclo activo mensual o anual

Pasos del flujo

- ? Stripe renueva la suscripcion y envia invoice.paid y/o customer.subscription.updated.
- ? El webhook procesa el evento en StripeSubscriptionHandler.
- ? ApplyReducerAsync y SubscriptionReducer mantienen al usuario en active o paid-through.
- ? AccessOrchestrator confirma acceso Full.
- ? El timer de reminders evalua luego las ventanas elegibles de auto-renew.

Que validar

- El usuario sigue en active con acceso Full.
- No se activa payment recovery.
- No se activa save-before-churn mientras auto-renew siga activo.

3. Auto-Renew Reminder Journey

Reminders previos a la renovacion automatica

Pasos del flujo

- ? SubscriptionReminderTimer ejecuta SubscriptionReminderService diariamente.
- ? El servicio lee el estado local proyectado y aplica prioridad de journeys.
- ? Si el usuario esta activo, con auto-renew ON y sin recovery ni cancel_at_period_end, entra en auto-renew.
- ? SubscriptionReminderEvaluator calcula si esta en una ventana elegible.
- ? Si la ventana aplica y no fue enviada antes, ManyChat recibe el flow correspondiente.

Que validar

- Mensual: 7d, 3d y 1d antes de renovar.
- Anual: 30d, 14d, 7d y 1d antes de renovar.
- La ventana de 1 dia usa framing positivo de continuidad.
- Se suprime si hay payment recovery o cancel_at_period_end.

4. Payment Recovery Inicial

Entrada al journey de recuperacion de cobro

Pasos del flujo

- ? Stripe envia invoice.payment_failed cuando falla el cobro recurrente.
- ? El webhook lo recibe y ApplyReducerAsync ejecuta el reducer.
- ? El usuario entra en past_due o grace segun las reglas actuales.
- ? Se guarda PaymentRecoveryStartedAtUtc en la proyeccion local.
- ? ManyChat recibe el flow inicial de recovery y el estado recovery_active.
- ? Payment recovery pasa a tener prioridad maxima sobre los otros journeys.

Que validar

- SubscriptionStatus delinquent o grace segun el reducer.
- PaymentRecoveryStartedAtUtc poblado.
- Flow inicial disparado solo una vez por episodio.
- Auto-renew y save-before-churn suprimidos.

5. Payment Recovery Follow-up Diario

Recordatorios mientras el usuario siga en recovery

Pasos del flujo

- ? El timer diario ejecuta SubscriptionReminderService.
- ? El servicio encuentra usuarios con PaymentRecoveryStartedAtUtc.
- ? Valida que el estado siga siendo delinquent segun la proyeccion local.
- ? Calcula el dia del episodio recovery.
- ? Si corresponde y no fue enviado antes, manda el follow-up de ManyChat.

Que validar

- Los follow-ups solo salen mientras el recovery siga activo.
- No hay duplicados por ventana o por episodio.
- Se detienen cuando el usuario sale de recovery.

6. Actualizacion de Metodo de Pago

Self-service seguro via Stripe Customer Portal

Pasos del flujo

- ? El usuario pulsa Actualizar metodo de pago desde ManyChat o frontend.
- ? La app llama al endpoint payment-method-update-link del backend.
- ? El backend valida ownership y crea una session de Stripe Customer Portal en payment_method_update.
- ? Devuelve la URL segura de Stripe.
- ? El usuario abre Stripe y actualiza su tarjeta.

Que validar

- Solo el dueno de la suscripcion puede abrir la session.
- No cambia el lifecycle local por si sola.
- No marca recovered sin invoice.paid.

7. Recovery Webhook-Driven por customer.updated

Retry automatico sin depender del return URL

Pasos del flujo

- ? Tras actualizar la tarjeta, Stripe emite customer.updated.
- ? El webhook procesa customer.updated y detecta cambio relevante de payment method.
- ? StripeSubscriptionHandler resuelve al usuario por StripeCustomerId.
- ? Si el usuario sigue en payment recovery segun la proyeccion local, el backend intenta invoice.pay.
- ? ManyChat puede recibir un estado operativo intermedio.
- ? El lifecycle todavia no cambia en este paso.

Que validar

- Solo hace retry si recovery sigue activo.
- No crea un state machine paralelo.
- No usa customer.updated como confirmacion final de negocio.

8. Retry Manual de Factura Abierta

Fallback o accion manual desde app

Pasos del flujo

- ? La app llama al endpoint retry-payment del backend.
- ? El backend valida ownership y consulta Stripe.
- ? Busca un open invoice para la suscripcion.
- ? Si collection_method es charge_automatically, ejecuta invoice.pay.
- ? Si recovery sigue activo localmente, sincroniza estado operativo a ManyChat.

Que validar

- No falla si no hay open invoice.
- No marca recovered por si solo.
- No envia estados de recovery si el episodio ya termino.

9. Confirmacion Final de Recuperacion

Salida oficial del journey de recovery

Pasos del flujo

- ? Stripe cobra con exito la factura y envia invoice.paid.
- ? El webhook procesa invoice.paid.
- ? El reducer saca al usuario de recovery y lo devuelve a active.
- ? PaymentRecoveryStartedAtUtc se limpia.
- ? AccessOrchestrator devuelve acceso Full.
- ? ManyChat recibe el estado recovered y se apagan los reminders de recovery.

Que validar

- SubscriptionStatus = active.
- PaymentRecoveryStartedAtUtc = null.
- AccessMode = Full.
- ManyChat = recovered.

10. Cancel at Period End

Cancelacion programada sin revocar acceso antes de tiempo

Pasos del flujo

- ? El usuario solicita cancelar al final del periodo.
- ? CancelSubscriptionAtPeriodEndUseCase valida ownership y actualiza Stripe.
- ? El webhook recibe el cambio y el reducer actualiza la proyeccion local.
- ? El usuario sigue activo hasta CurrentPeriodEndUtc.
- ? Auto-renew deja de aplicar y entra save-before-churn.

Que validar

- StripeCancelAtPeriodEnd = true.
- Acceso sigue Full hasta la fecha final.
- No hay early revocation.

11. Save-Before-Churn Journey

Recordatorios cuando la cancelacion ya esta programada

Pasos del flujo

- ? El timer diario evalua usuarios con `cancel_at_period_end = true`.
- ? `SubscriptionReminderEvaluator` usa `CurrentPeriodEndUtc` para calcular la ventana.
- ? Si la ventana aplica y no se envio antes, ManyChat recibe el flow ending soon o reactivate.
- ? Auto-renew se mantiene suprimido mientras siga esta condicion.
- ? Si tambien hay recovery, payment recovery gana por prioridad.

Que validar

- Mensual: 7d, 3d y 1d antes del fin real.
- Anual: 30d, 14d, 7d y 1d antes del fin real.
- No hay dual-send con auto-renew ni con recovery.

12. Deletion o Fin Final

Cierre terminal de la suscripcion

Pasos del flujo

- ? Stripe envia customer.subscription.deleted o evento final equivalente.
- ? El webhook procesa el evento.
- ? El reducer mueve al usuario al estado terminal.
- ? AccessOrchestrator bloquea acceso.
- ? Se limpian estados transitorios como grace o recovery.
- ? No deben salir mas journeys ni reminders.

Que validar

- Estado terminal correcto.
- AccessMode = Blocked.
- Sin auto-renew, sin save-before-churn y sin payment recovery.

13. Retry y Replay Safe Behavior

Reintentos sin mensajes viejos o contradictorios

Pasos del flujo

- ? Si una accion de ManyChat falla de forma retryable, se guarda en failed actions.
- ? FailedActionRetryService la reintentada mas tarde.
- ? Antes de reenviar, valida el estado local actual del usuario.
- ? Si el episodio ya termino o un journey superior tiene prioridad, suprime el envio.
- ? Solo reenvia si el estado sigue vigente.

Que validar

- No se reenvian reminders viejos.
- No se reenvia recovery si ya termino.
- No se envia auto-renew si recovery o churn ya tienen prioridad.

Checklist para Tests Integrales

Usa este bloque como base comun de QA para todos los escenarios.

- ? Verificar evento o request de entrada correcto.
- ? Verificar proyeccion local del User despues del flujo.
- ? Verificar acceso efectivo calculado por AccessOrchestrator.
- ? Verificar estado, tags, custom fields o flows enviados a ManyChat.
- ? Verificar suppression rules y prioridad entre journeys.
- ? Verificar dedupe, replay-safe behavior y out-of-order protection.
- ? Verificar que el lifecycle final solo cambie por webhook y reducir.
- ? Verificar que invoice.paid siga siendo la confirmacion final de recovery.

Documento generado para uso interno, documentacion funcional y pruebas end-to-end.